

Users Pay Twice: The Hidden Energy Cost of Web Advertising

Samuel Péliissier
CentraleSupélec, Inria, IRISA
Rennes, France

Naif Mehanna
Independent
Lille, France

Sterenn Roux
Univ. Rennes, CNRS, Inria, IRISA
Rennes, France

Quentin Perez
INSA Rennes, Univ. Rennes, CNRS,
Inria, IRISA
Rennes, France

Walter Rudametkin
Univ. Rennes, CNRS, Inria, IRISA, IUF
Rennes, France

Johann Bourcier
Université de Pau et des Pays de
l'Adour, E2S UPPA, LIUPPA
Anglet, France

Pierre Laperdrix
CNRS, Univ. Lille, Inria
Lille, France

Abstract

Since the first online ad in 1994, advertising has grown into a vast ecosystem delivering billions of ads daily. Advertisements are everywhere on the Web: search engines promote results, most websites display ads, some require users to accept ads as a condition for access, video streaming services fund their infrastructure through an increasing volume of ads, and much of the gaming industry has adopted ad-based revenue models. In exchange for free access to a wide range of content, web users sacrifice their privacy and pay with personal data to enable targeted marketing, a trade-off widely studied in literature. We argue that the ad ecosystem imposes an additional, overlooked, cost on web users: energy consumption.

In this paper, we present a large-scale analysis of the client-side energy consumption of ads, including their associated tracking mechanisms. We design a robust methodology aimed at realistically modeling user behavior and monitoring CPU activity. Through measurements of 724,994 website visits across diverse devices, we study the energy implications of consenting to tracking and of blocking ads through browser and network-based software. We find that consent via cookie banners increases energy consumption by a median 2.57% across all websites and devices. We also show that websites relying on real-time bidding increase median energy consumption by 33.98%. Finally, although ad-blocking solutions consume energy due to their filtering processes, we observe a median reduction of 9.62% in client-side energy consumption when using uBlock Origin.

CCS Concepts

• **Security and privacy** → **Social aspects of security and privacy**; • **Information systems** → **Online advertising**; • **General and reference** → *Measurement*.

Keywords

Web, Privacy, Tracking, Energy Consumption

ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only. Request permissions from owner/author(s). *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792414>

ACM Reference Format:

Samuel Péliissier, Naif Mehanna, Sterenn Roux, Quentin Perez, Walter Rudametkin, Johann Bourcier, and Pierre Laperdrix. 2026. Users Pay Twice: The Hidden Energy Cost of Web Advertising. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792414>

1 Introduction

There are over 5.56 billion Internet users in the world [11]. It was estimated that 120 zetta bytes of data were created during 2023 [10]. The demand for new services and content to be delivered at the highest quality and lowest possible latencies is high. Interestingly, much of the Internet is virtually free for users to access, financed indirectly through advertising. To increase revenue, the advertising industry has evolved from simple banners to a complex ecosystem that relies on tracking and profiling users to better target them with personalized ads [63, 64]. This comes at the cost of personal data and has raised serious privacy concerns, leading to the adoption of several legal frameworks, such as the GDPR in the European Union [67], or the CCPA in California (US) [21]. Similarly, multiple studies have focused on various tracking techniques, such as browser fingerprinting [44], cookie synchronization [20], server-side tracking [15] or consent management [32]. However, an increasingly relevant *hidden cost* of ads is their energy consumption. During much of the same period that online advertising developed, the consequences of climate change have become better understood and more visible. According to the United States Environmental Protection Agency, global greenhouse gas emissions increased by 43% between 1990 and 2015 [3]. Data centers host most of the content that is accessed online and account for more than 1% of the global electricity demand [2]. This has led many studies to focus on assessing and optimizing data center energy consumption [40–42, 69].

This study tackles another perspective. Through a user-centric approach, we focus on the additional costs of ads and tracking *directly in the browser*. For example, generating a browser fingerprint for tracking purposes requires numerous calls to different APIs to extract the browser version, screen resolution, fonts, etc. [44]. These operations induce an energy cost for the client: they require CPU cycles, GPU calculations, and RAM to temporarily store the data. In addition, the code must be downloaded by the client and the results transmitted to a server for further processing by the

advertiser (e.g., to re-identify the user) [44]. Again, this comes at a cost to the user. This additional hidden aspect of ad-related tracking schemes is seldom addressed in the literature. We argue that it deserves greater academic attention since there is widespread use of tracking technologies, it affects all users on the Web, and environmental concerns are growing [31, 39–42, 56, 69].

In this paper, we present real-world measurements of *client-side energy consumption attributed to ads and tracking*. In particular, we make the following contributions:

- (1) We design a robust energy measurement pipeline for web browsers that simulates realistic user behavior, offering a significant improvement over the state-of-the-art.
- (2) Using a large-scale dataset of 724,994 website visits, we characterize the energy impact of tracking and advertising under various conditions, including the use of browser and network-based ad-blocking software, cookie consent choices, and the deployment of client-side header bidding.

2 Background & Motivations

Our study builds on previous work in web advertising with a focus on the impact of ad blockers on energy consumption. It is complemented by a client-side analysis of Real-Time Bidding.

2.1 Online advertising

Ads generated over \$258.6B in 2024 [34], representing a 15% increase over the previous year, highlighting the fast pace at which the advertising ecosystem is developing. Advertising is present in nearly every aspect of the Web. From search engines to video streaming platforms, resource-intensive ads are displayed, requiring various actors to continuously build new infrastructure to meet growing demand. In 2015, it was estimated that more than 18% of HTTP requests made by web pages were related to advertising [59].

Modern advertising processes are complex and often combined with extensive user profiling to better target their advertisements, with much of the data obtained through web tracking techniques [63, 64], unfortunately increasing resource usage. Web tracking comes in various forms. The most traditional method, cookie-based tracking, has evolved significantly since its first mention in mainstream media in 2001¹. It now involves intricate systems with deeply entangled cookie-sharing mechanisms [13, 55]. More recently, stateless tracking mechanisms have emerged. For instance, browser fingerprinting [43, 44, 49] identifies users through details about their hardware and software configurations collected via browser APIs. Although some of this information, such as the user agent, is easily accessible, more robust techniques rely on attributes such as canvas fingerprinting [49] or GPU fingerprinting [43], which are significantly more resource intensive. These tracking mechanisms introduce additional strain on the user's device, directly contributing to higher energy consumption.

2.2 Ad bidding

Real-Time Bidding (RTB) [33], introduced in the late 2000s, enables advertisers to purchase individual ad impressions via auction on ad exchange platforms. In this process, publishers make their ad space

available, advertisers evaluate the web user profile provided by the exchange, and bids are submitted through *Demand-Side Platforms*. The highest bid above the publisher's *floor price* wins, and the corresponding ad is displayed. RTB has since become a central component of the online advertising ecosystem [18, 52]. *Header Bidding* [65] emerged more recently as a method where publishers directly contact multiple ad exchanges or Supply-Side Platforms in parallel, improving transparency and giving publishers greater control over which exchanges participate in the auction.

Adoption of header bidding has grown rapidly: in 2019, Pachi-lakis *et al.* [52] found that over 14% of Alexa Top-35K websites used client-side header bidding. To do so, they detect the inclusion of the `prebid.js`² library, an open source wrapper for publishers to connect with their respective ad networks. Similarly, the `ads.txt` standard combats ad fraud by letting publishers list authorized exchanges in a file hosted on their website. Bashir *et al.* [18] reported that in April 2019, 19.7% of Alexa Top-100K sites hosted a valid `ads.txt`, while more than 60% of those using RTB hosted a `ads.txt` file. For our study, header bidding is particularly relevant: it explicitly signals a website's intent to host ads, shifts additional operations to client devices, and is associated with more extensive data collection [18]. As the latest widely adopted form of online advertising, it is important to understand how header bidding impacts energy consumption.

2.3 Ad-blocking techniques and impact of advertisement

To avoid overly intrusive ads, mitigate malicious content distributed by ads [58], and reduce privacy leaks, many users turn to palliative solutions that block the advertising and tracking ecosystem [66]. *Ad blockers* are typically installed as browser extensions or, less frequently, deployed at the network level as DNS resolvers. While the network-based solutions offer broader protection across all devices on a network, they typically lack fine-grained filtering capabilities. As seen in Figure 1, browser extensions can leverage contextual information (e.g., the DOM, query parameters) and real-time script analysis for more precise blocking.

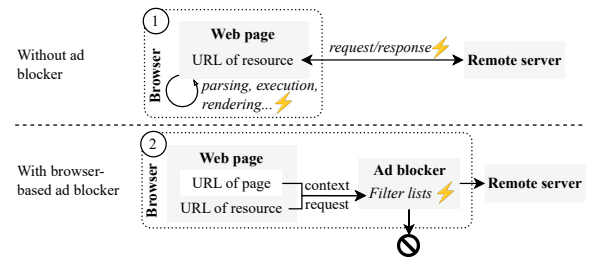


Figure 1: Energy-consuming operations (1) without and (2) with a browser-based ad blocker.

The current adversarial stance towards advertising, and the growing number of users of ad blockers, is arguably pushing Google to reduce the capabilities of browser extensions in Chrome. In particular, forced migration from Manifest V2 to V3 [54] limits the permissions of updated extensions, thus restricting their ability to

¹<https://www.nytimes.com/2001/09/04/business/giving-web-a-memory-cost-its-users-privacy.html>

²<https://prebid.org/>

detect and block advanced tracking and advertising techniques.³ Ad blockers play a significant role in our user-centric study. First, they provide a practical way to estimate the impact of ads and tracking on the Web by enabling comparisons between visits with and without them. Second, in addition to protecting privacy, ad blockers help avoid energy-consuming operations such as network requests, script parsing and execution, and ad rendering. However, as shown in Figure 1, ad blockers rely on large filter lists containing numerous rules, which can result in CPU-intensive processing in the browser. This raises an important question when optimizing for energy consumption. Is it worthwhile for end users to block advertisements and tracking to save energy?

Few studies have examined energy consumption on the Web. Some have explored the impact of tracking on smartphone battery life [24, 28]. However, these focus on in-app tracking, which differs significantly from web tracking [24], and measure system-wide consumption, lacking granularity [28]. In addition, the advertising landscape has evolved considerably in the past decade. On desktop systems, Pearce [56] showed in 2020 that uBlock Origin [7], a popular ad-blocking extension [66], reduced page load times by over 28% and could save users more than 100 hours of electricity per year. However, this study considered only 10 websites, focused on load and visit times, and estimated rather than directly measured energy consumption. A similar-scale experiment by Khan *et al.* in 2024 [39] examined several ad-blocking extensions and reported that ad blocking generally reduces energy consumption but can increase it on some sites. Their analysis, however, was limited to CPU-level measurements on a single device, a small set of 10 websites, and omitted page interactions such as clicking or video playback, which significantly affect energy use.

Finally, Gonzalez *et al.* released CARBONTAG [31] in 2023, a framework that *estimates the carbon emissions of ads*. Their approach uses a trained ML model to approximate the emissions from rendering ads and achieves high accuracy. However, it does not capture the *actual energy cost paid by web users* and faces several limitations, making it difficult to generalize to real-world settings. First, it relies on CODECARBON [8], a system-wide CPU energy measurement tool that mixes operating system and browser activity. Second, it considers exclusively iframe-based ads, ignoring any resources outside the iframe, such as scripts, which are CPU-intensive and central components of the modern ad ecosystem. Third, its handling of cookies during ad collection is unclear, despite cookie behavior being central to tracking, as we demonstrate in our work.

These works show a gap in the literature, not only regarding the impact of advertising on energy consumption but also in the methodologies used to measure it. Our work addresses this by leveraging recent state-of-the-art energy measurement software, POWERAPI [29], which enables fine-grained process-level measurements with better robustness and precision than other options [36]. Furthermore, we place special emphasis on simulating realistic user behaviors to accurately estimate the impact of ads and tracking on the energy consumption of web browsers on 5000 websites, an order of magnitude above the state-of-the-art [39, 56].

³For instance, blocking via response headers is not possible with Manifest V3 [https://github.com/uBlockOrigin/uBOL-home/wiki/Frequently-asked-questions-\(FAQ\)#filtering-capabilities-which-cant-be-ported-to-mv3](https://github.com/uBlockOrigin/uBOL-home/wiki/Frequently-asked-questions-(FAQ)#filtering-capabilities-which-cant-be-ported-to-mv3)

3 Measurement methodology

We propose an approach that robustly measures the energy consumption of web browsing sessions while inferring the resource usage related to advertisement and tracking. Our design relies on multiple crawl variants (§ 3.1) and enables fine-grained analysis of CPU processes (§ 3.2) with various ad-blocking techniques (§ 3.3). To ensure that our measurements closely reflect the experience of a typical web user, we leverage client-side instrumentation to generate realistic browsing behaviors (§ 3.4). The entirety of the code, including deployment automation, is available at <https://doi.org/10.5281/zenodo.18309442>.

3.1 Crawl variants

Crawls in our experiments correspond to lists of web pages visited under different settings, called *crawl variants* (Figure 2). Each variant combines two factors: the presence of an ad blocker and user consent to tracking. By comparing each variant to a baseline crawl, we estimate the relative impact of these factors on energy consumption. For example, to isolate the effect of ad blocking, we compare a baseline crawl without an ad blocker to a variant using one, with consistent cookie consent across both crawls.

Each page list is derived from the top 5,000 Tranco websites (§ 3.4.1), with two random clickable links selected from each front page to increase dataset representativeness [61]. This yields 15,000 pages per list (5,000 front pages and 10,000 internal pages). We then sequentially visit the same pages under different settings, e.g., with or without cookie consent or ad blocking (§ 3.3). Crawl variants are performed identically across all devices. To mitigate measurement noise, we repeat the process up to five times with different random page lists. To avoid duplicate visits (once during page list creation and again during crawling), we initialize PRNGs with the same seed on each client device, ensuring that all devices visit the same set of pages across variants. Results are extracted by comparing each client device’s baseline crawl with its corresponding variant for the same seed. Unless stated otherwise, results are aggregated across all seeds and devices. For fairness, we exclude websites where sampled internal URLs differ between variants (e.g., due to site updates).

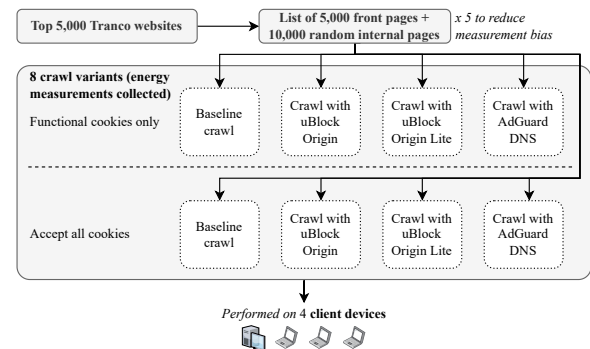


Figure 2: Conceptual process for selecting web page lists and crawl variants.

3.2 Robust energy measurement

We build our solution on top of state-of-the-art measurement software [29, 36] and take various steps to limit measurement errors.

3.2.1 Architecture. At a high level, our approach is based on a client-server architecture. A client device hosts an automated browser that crawls a set of websites (*i.e.*, a crawl variant). The client device uploads relevant data to a distant server, including raw system performance metrics, network traffic information, and miscellaneous data about the websites (*e.g.*, cookies, HTTP requests and responses). More precisely, only the data relevant to a specific browser process is measured and saved. To achieve this, we leverage a state-of-the-art energy consumption software, POWERAPI [29]. Its software sensor collects raw metrics from the Running Average Power Limit (RAPL) interface [35] on the client device, uploads them to a database on the server, and finally estimates energy consumption through a dynamic machine learning power model [30]. By design, POWERAPI precisely reports the CPU consumption, including integrated graphics [30, 36], of any process placed in a Linux control group [1]. Thus, the software sensor deployed on a client device only monitors the browser process, ignoring the rest of the system. Finally, to automate the browser itself, we use Puppeteer and, due to its large market share, the latest version of Chrome. We apply Rebrowser patches to reduce bot detection.⁴

3.2.2 Increasing robustness. Empirical energy measurements on the Web are subject to various sources of error, *e.g.*, due to high system load when measuring a specific program or faulty websites. We take multiple steps to mitigate these issues.

Hardware and software stability across crawls. To mitigate deployment discrepancies, client devices follow the same automated installation process and they are not updated between crawls. Unless specified, their settings, including browser and extension settings, remain at their defaults. When laptops are used as client devices, their lids are kept closed so that the built-in displays remain off, ensuring consistency across hardware. In any case, experiments leverage a virtual display server, *i.e.*, display computations are still done by the CPU.

Computation offloading. Overloading the client device's CPU may negatively affect energy measurements [23]. To address this, we empirically confirm that parallelizing crawls on a single machine does not impact the stability of measurements. More precisely, we launch at most $n - 2$ crawling threads, where n is the number of available CPU cores. This leaves enough computational power for other processes (*e.g.*, the software sensor) without overloading the CPU. Additionally, we offload all computation-heavy tasks unrelated to browser automation to a remote server. For example, the software sensor is only responsible for uploading raw sensor data. The conversion into energy consumption values is done server-side.

Ignoring erroneous websites. We refrain from directly comparing the total accumulated energy consumption between two crawls variants. Since each crawl includes thousands of websites, there is no guarantee they successfully visit all of them. To avoid skewed results, we report energy consumption based solely on website-to-website comparisons. In other words, if a website visit fails during one of the two compared crawls, it is excluded from results. We go even further by considering a website visit to be erroneous if *any* of its page visits fails (*e.g.*, due to occasional timeouts).

Reducing timing variations. Energy consumption is computed based on the duration of a website visit. Longer visits naturally

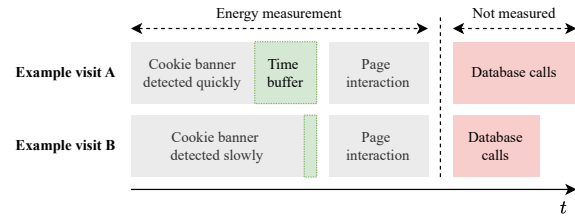


Figure 3: Improving energy measurement robustness to limit timing variations between visits.

result in higher energy use. Therefore, we ensure that two visits on the same website last approximately the same amount of time. Due to the heterogeneity of visited websites, we assume this design is better than forcing a fixed timer for each visit, which may lead to unpredictable loading interruptions and thus unstable measurements. As shown in Figure 3, we reduce timing discrepancies between visits using two strategies. First, we enforce fixed durations for specific operations. For example, interacting with a cookie banner (§ 3.4.4) may take anywhere from a few milliseconds to several seconds depending on the website and current CPU load. To prevent such a variability from affecting visit durations, we introduce a time buffer. Second, network conditions can delay database interactions, potentially extending *some* visit times. To avoid this, we move all database operations out of the window of energy measurements.

3.3 Blocking ads and tracking

As explained in Section 2, web users can opt for two ad-blocking techniques: browser or network-based solutions. Our goal is *not* to study the relative blocking effectiveness of ad blockers. However, the recent migration from Manifest V2 to V3 offers a unique opportunity to measure its impact on energy consumption. We therefore select two flavors of the same highly popular [66] browser extension: the newer *uBlock Origin Lite*, which uses Manifest V3 APIs, and *uBlock Origin*, which uses the more powerful yet deprecated Manifest V2 APIs [7]. Unless specified otherwise, all results presented in this paper are obtained with the default settings of *uBlock Origin Lite*. This provides a conservative estimate, closely related to what a modern user would encounter on the Web. Although *uBlock Origin* has now been removed from the Chrome Web Store, we manually installed it for our experiments. This gives insight on the energy consumption of a more capable ad blocker. DNS resolvers are an alternative solution to block ads and trackers. Instead of using their default DNS resolvers (*e.g.*, their ISP's DNS), users can set their DNS to an ad-blocking DNS, which answers with an empty reply when queried for the IP address of a known advertising domain. We chose *AdGuard DNS* due to its popularity, free tier, and its use of similar filter lists as *uBlock Origin Lite* [9]. To avoid DNS leakage and to quickly change configurations, we set each crawler's DNS resolver through a network namespace.

All ad-blocking solutions are set up with default parameters and behave as they would for a typical web user. To provide additional statistics, we introduce a post-processing step on the server, where we tag ad and tracking-related requests captured during our crawls. As in the literature [25], this is done using Brave's *adblock-rs* module along with five up-to-date filter lists.⁵

⁴<https://github.com/rebrowser/rebrowser-patches>

⁵EasyList, EasyPrivacy, uBlock Origin Ads, uBlock Origin Privacy, and AdGuard Tracking; latest versions were automatically downloaded on the day of the experiments.

3.4 Simulating realistic user behavior

To measure the hidden energy cost of ads and tracking mechanisms as experienced by real-world web users, we design a process that simulates their behavior. Although this is a complex task [27], the methodology described below combines multiple steps and improves the state-of-the-art.

3.4.1 Top lists. The first step is to select a relevant set of websites to visit. We use the Tranco list for its completeness, relevance, and lack of censorship [57, 62]. Previous studies have shown that the top 10,000 websites account for "75–80% of desktop traffic" [61]. While users might not visit all of these websites, they are likely to visit *some*, making our analysis relevant to most, if not all, web users. Due to the time required to crawl these many websites multiple times, we limit our study to the top 5,000 (Tranco list version *Lj9Z4*).

3.4.2 Preparing profiles. Previous research shows that advertisers are more likely to display ads to users with an established browsing history and profile [19, 26, 50, 51]. Although most regular web users already have such a profile (unless they clean their browsing state), we need to create one for each crawl. Duplicating profiles across multiple devices or IP addresses risks bot detection or errors, e.g., from shared session cookies. For comparison purposes, we build profiles on each client device by sequentially visiting the same set of pages. This ensures that each client device has a unique profile (e.g., unique cookies) based on the same navigation history.

More precisely, each client device participating in a crawl starts from a clean state, i.e., no cookies and an empty cache. We select 100 random websites from the top 5,000 of the Tranco list [57], crawled in the same order by each client device. During this phase, energy consumption is not measured; devices accept cookies and do not block ads. For each website, the client device visits the front page and two randomly selected pages (i.e., the same pages for all client devices), for a total of 300 page visits per profile. Each profile is cloned for subsequent energy-measuring crawl variants on that client device but not shared across client devices. Since we leverage IP addresses already associated with legitimate, user-generated traffic (§ 4), we start crawling immediately after profile generation.

3.4.3 On-site behavior. It is currently unfeasible to simulate complex, realistic user behavior on *all* websites due to the high variability of both websites and users [27]. Technically, this would require developing methods to authenticate on thousands of websites and interacting with complex features, such as watching videos, uploading content, or purchasing products and completing the checkout process. In this work, we focus on a subset of realistic behaviors, limited to non-authenticated users visiting a few pages per website, typically to retrieve information. This approach follows state-of-the-art methodologies [46, 48, 53] and does not aim to represent all possible interactions with ads, such as watching videos (§ 6). More precisely, we instrument the following process: (1) load the front page of a given website (fail after a timeout of 20 seconds); (2) interact with its cookie banner (§ 3.4.4) and fail after a timeout of 3 seconds if the following network calls do not stabilize; (3) scroll down 10 times over 6 seconds or wait at the bottom of the page if reached; (4) select 2 random clickable links from the front page (the same pages for all client devices and subsequent crawl variants); (5) repeat the process for these pages by opening and closing a

new tab each time. In our dataset, each successful website visit, corresponding to three page visits, lasts 68 seconds on average.

3.4.4 Handling cookies. Cookie banners are a central component of the tracking ecosystem, as consenting to them allows websites and their third-party partners to track users. On some websites, users *must* interact with the banner to access the page's content [22]. Since our goal is to simulate realistic user behaviors, automating cookie consent is required. We define two distinct cookie settings for our crawls: "Accept all cookies" and "Functional cookies only". The former consents to cookies, targeted ads, and tracking, while the latter is expected to minimize cookies and explicitly refuse targeted ads and tracking. Each crawl is performed using either one of these settings. Following prior work on cookie banners [37], we use a multi-step pipeline to interact with the banners by analyzing the DOM with the *EasyList Cookie List* and clicking on interactive elements based on common keyword variations. For completeness, we further detail this process in Appendix A. While a more advanced technique based on natural language processing has recently been published [22], their solution demands substantial resources compared to our approach and would not scale to the number of crawls we perform. We consider our ~70% success rate in banner interactions to be sufficiently high for this analysis.

4 Experimental setup and dataset

For our experiments, we rely on four distinct real-world client devices: three laptops and one desktop, detailed in Appendix B. This selection represents a diverse pool of consumer-grade devices, an improvement over previous setups [31, 39, 56]. As each client device includes an integrated GPU, POWERAPI correctly measures all their computationally intensive tasks. To ensure fair comparison of energy consumption across client devices, they all run identical software stacks: an up-to-date Ubuntu Linux 24.04 (kernel 6.11.0) and the latest version of Chrome (136.0.7103.59).⁶ Client devices are connected from three different vantage points within France: two university networks (*Université de Rennes* and *Université de Lille*) and a residential home. These provide well-reputed IP addresses, which helps reduce the risk of being detected as bots [14].

We crawled websites from 6 March to 22 May 2025, achieving a 73% success rate over 724,994 website visits, with no significant variation across client devices or over time. Only successfully crawled websites are included in the following results. If any page of the website fails, the entire website is considered as failed (§ 3.2.2). As detailed in Section 3.1, we reproduce crawls multiple times with different random pages lists to reduce statistical bias. In practice, due to the difference in parallelization power between client devices, crawls are replicated between 3 times (Dell Latitude 7490) and 5 times (all other client devices) for every crawl variant (i.e., with and without ad blockers and consent). Combining all measurements, these crawls generated more than 300,000,000 HTTP requests.

Due to the dynamic nature of the power meter used by POWERAPI [29, 30], and despite the steps towards robustness described in Section 3, some measurements exhibit abnormally high or low values during model self-calibration. To ensure data quality, we remove outliers beyond 1.5 times the interquartile range (IQR). They

⁶Manifest V2 is scheduled for deprecation in Chrome version 138, allowing both *uBlock Origin* and *uBlock Origin Lite* to be installed on the same browser version.

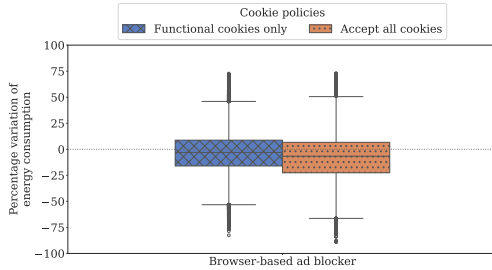


Figure 4: Energy consumption of the crawl variants "browser-based ad blocker with *Accept all cookies*" and "browser-based ad blocker with *Functional cookies only*" compared to the baseline crawls.

account for only 2.9% of the total data points, leaving a large and interpretable dataset.

5 Results

In this section, we present the results of our crawling process. Blocking through browser extensions requires CPU cycles on the client device to analyze network requests and block ads. Likewise, blocking lists are imperfect (*i.e.*, some ads and tracking scripts slip through the gaps). Hence, results presented in this section offer a lower-bound estimate of the actual energy cost of ads and tracking on the Web. Unless specified otherwise, results are presented based on crawls done on all client devices using the *uBlock Origin Lite* browser extension. We further demonstrate the statistical significance of the results in Appendix C.

5.1 Advertisement on the Web

5.1.1 Impact of cookie banners. We find that consenting to tracking by accepting all cookies in cookie banners increases energy consumption by a median of 2.57% compared to accepting functional cookies only. The increase is easily explained by the greater resource usage required by the browser to handle ads and additional scripts: a median 8.5 extra requests are sent when accepting all cookies, 2 of which are detected as advertising and tracking. Although ads and tracking contribute to energy consumption, our measurements represent a conservative estimate of their actual impact. First, non-targeted ads can be loaded even without consenting to tracking. Second, websites may not respect users' consent [45]. Third, tracking scripts can be downloaded and parsed *before* any interaction with the cookie banner, potentially impacting users who accept functional cookies only. Therefore, we further study possible reduction strategies w.r.t. ads; namely, the use of ad blockers.

Figure 4 illustrates the variation of energy consumption when crawling websites with an ad-blocking browser extension (*uBlock Origin Lite*) compared to a baseline without ad blocker. Blocking ads and tracking generally reduces the energy consumption of client devices, with a -3.01% median decrease across all measurements when accepting functional cookies only, and a median of -6.64% when accepting all cookies. Additionally, functional cookies should only enable specific website features that are unrelated to targeted advertising or tracking. Therefore, when measuring the energy consumption of websites while accepting functional cookies only, we expected a slight increase in energy consumption due to the

Table 1: Successfully visited websites per category, in a single crawl, including the total number of requests and the ratio of ad-based requests.

Category	Websites	HTTP Requests	Ad ratio
News	337	188,463	41.94%
Search Engines/Portals	144	36,400	37.79%
Entertainment	127	54,949	35.05%
Business/Economy	240	97,028	30.11%
Technology/Internet	718	232,933	26.97%
Others	1,540	626,519	26.14%
Shopping	161	95,934	26.05%
Government/Legal	118	27,112	17.75%
Pornography	106	28,858	16.39%

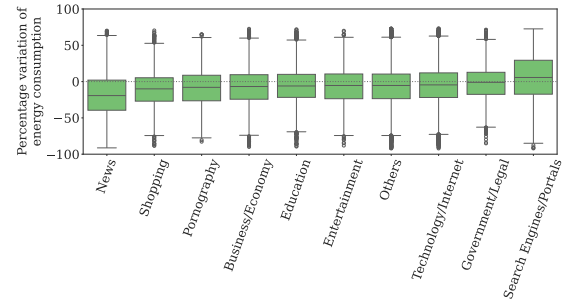


Figure 5: Energy consumption for the top 10 website categories of the crawl variant "browser-based ad blocker with *Accept all cookies*" compared to the baseline crawl.

additional CPU cycles required by the browser extension to analyze all of the non-tracking requests. However, we hypothesize that the observed decrease in energy consumption may be due to (a) *uBlock Origin Lite* blocking scripts that are pre-loaded on the page (but potentially not used until consent is provided), (b) filter-lists blocking additional resources such as aggregated analytics that the GDPR allows to collect without consent, and (c) websites not respecting user consent [45].

5.1.2 Variability per category. Table 1 shows the websites successfully visited in a single crawl, per category. A significant percentage of requests in our dataset are related to advertising and tracking, although this proportion varies across categories. Unsurprisingly, the *News* category, whose business model relies heavily on advertising, exhibits the highest ratio of advertisement-related requests at 41.94%. It also has the highest absolute number of advertising-related requests among all categories (except *Others*).

The absolute number of requests reflects the presence of ad and tracking-related technologies, but it does not capture their impact on energy consumption. To address this, we compare the energy consumption of baseline crawls without an ad blocker to that of crawl variants using a browser-based ad blocker, with *all cookies* accepted in both cases. As shown in Figure 5, websites in the *News* category exhibit a substantial reduction in energy consumption (median: -19.28%), consistent with previously observed trends. At the other end of the spectrum, *Government/Legal* websites show only a modest decrease (median: -1.01%), reflecting their limited use of targeted advertising and tracking technologies.

5.1.3 Overhead of header bidding. On average, out of the 3005 successfully visited domains per crawl, we observe that 346 (12%)

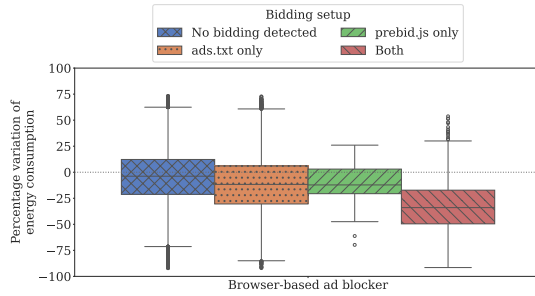


Figure 6: Energy consumption per header bidding setup type for the crawl variant "browser-based ad blocker with *Accept all cookies*", compared to the baseline crawl.

websites include `prebid.js`, 933 (31%) websites host `ads.txt` file, and 337 (11%) websites combine both approaches. This is in line with reported number in previous works [18]. We compare the energy consumption for each header bidding setting when accepting all cookies to that of the baseline crawl and report the results in Figure 6. Unsurprisingly, websites displaying targeted ads through bidding report a greater decrease in energy consumption when an ad blocker is running. For instance, websites including `ads.txt` show a -12.19% median decrease, contrary to -3.81% without bidding detected. When combining both `prebid.js` and `ads.txt`, we reach a -33.98% median decrease in energy consumption. Here, we measure the effect of ad blocking on the studied websites, not the intrinsic energy consumption of header bidding, which would require measurements significantly more fine-grained. The large decreases may partly reflect our sample: websites using programmatic ads and header bidding may display more ads and tracking than average. Finally, although `ads.txt` is widely deployed, these numbers likely represent a lower-bound estimate for ad-heavy websites, as some may use neither detected techniques.

5.2 Impact of ad-blocking techniques

So far, all results were obtained using the *uBlock Origin Lite* browser extension. As explained in Section 3.3, other means of blocking ads and tracking exist. Figure 7 showcases the effectiveness of two of these alternative setups, namely the *uBlock Origin* browser extension before its suppression from Google Chrome's store, and AdGuard's blocking DNS, labelled *Network-based ad blocker*. Due to technical difficulties when setting up *uBlock Origin*, now removed from the Chrome Web Store, the *Intel NUC 9i7QNB* client device is excluded from results in this section.

uBlock Origin demonstrates the greatest impact, reducing energy consumption by a median of -9.62% for *Functional cookies only*, compared to -3.57% for *uBlock Origin Lite*. Our hypothesis is that *uBlock Origin*'s detailed filters might consume as much as those used by *uBlock Origin Lite*, but block ads and tracking scripts much more effectively. Further work specifically measuring the performance discrepancies between both flavors of *uBlock Origin* should shed light on the exact reason.⁷ In any case, Google's decision to enforce the adoption of Manifest V3 and thus remove *uBlock Origin*

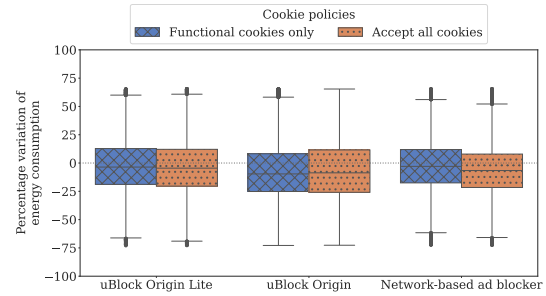


Figure 7: Energy consumption per ad-blocking technique for the crawl variants "ad blocker with *Functional cookies only*" and "ad blocker with *Accept all cookies*", compared to the baseline crawls.

from its store goes beyond potential privacy concerns, as it is currently responsible for a significant energy consumption increase. As shown in Figure 7, reduced ads and script downloads/executions outweigh the cost introduced by the browser extension in *most* cases. However, this trade-off cannot be expected for *all* websites due to specific behaviors that increase energy consumption. For instance, `www.linkedin.com` continuously sends POST requests to the `/li/tracking` endpoint when blocked, further increasing the number of CPU cycles associated with the browser extension's ad-blocking activity. For this specific website, energy consumption increases by a +12.13% median with an ad blocker.

In addition, *Network-based ad blocker* reduces energy consumption by a -6.63% median when accepting all cookies. By offloading the CPU-intensive blocking process to another client device, *Network-based ad blocker* outperforms *uBlock Origin Lite*. However, it cannot reach a similar impact as *uBlock Origin*, for two reasons. First, DNS-based filtering can only leverage the domain name, not the full context of a request (e.g., DOM, URL parameters), making it less effective compared to a full-fledged extension like *uBlock Origin*. Second, blocking at the DNS level introduces potential client-side errors that might generate additional energy consumption (e.g., missing JavaScript functions). This can be illustrated by visiting `uk.com`. With *Network-based ad blocker*, this website loads two scripts from `googletagmanager.com` but blocks POST requests to `google-analytics.com`, resulting in a console error. Alternatively, *uBlock Origin* blocks all tracking requests and leverages a scriptlet containing no-operation functions to prevent the website from breaking [16], thereby limiting errors and network retries. This difference in behavior is also reflected in energy consumption: we observe a median decrease of -4.51% with *Network-based ad blocker*, compared to a -17.33% decrease with *uBlock Origin*.

5.3 Variability per client device

Previous results aggregate measurements across all client devices, but energy consumption varies significantly between them. On average, a website visit consumes 0.13 Wh, 0.09 Wh, 0.07 Wh, and 0.04 Wh on the *Intel NUC 9i7QNB*, *Dell Latitude 7490*, *Lenovo Thinkpad T480*, and *Dell XPS 13 7390* respectively. All client devices use CPUs released in the same period (2017 - 2019, see Appendix B), with identical BIOS and C-state settings. However, they have different

⁷Anecdotally, non academic work have shown that *uBlock Origin* actually consumes less CPU resources than *uBlock Origin Lite*: <https://x.com/gorhill/status/1792648742752981086>.

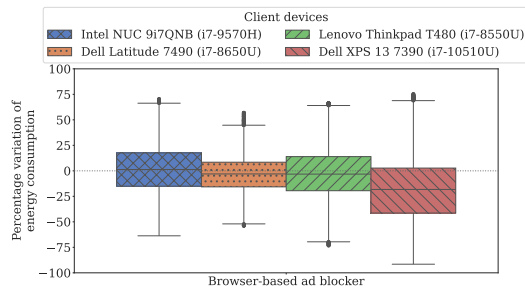


Figure 8: Energy consumption per client device for the crawl variant "browser-based ad blocker with *Accept all cookies*", compared to the baseline crawl.

CPU architectures, with varying degrees of dynamic power optimization. For instance, laptops are generally more optimized to aggressively save battery, switching C-states more often than NUCs. Hence, blocking ads and tracking on a laptop with a highly dynamic architecture may shutdown a core more easily, reducing energy consumption. Figure 8 illustrates this phenomenon when comparing crawl variants accepting all cookies to the baseline crawls. For instance, the stable *Intel 9i7QNB NUC* shows a +1.32% median increase in energy consumption when blocking ads, whereas the browser-based ad blocker introduces a -18.26% median decrease on the more dynamic *Dell XPS 13 7390* laptop.

6 Implications, limitations, and future work

Our study demonstrates that ads and tracking affect the energy consumption of web pages on users' devices. Ad-blocking software, while also consuming resources, is generally worthwhile, reducing median energy consumption by 9.62%. Although such an improvement may seem relatively low, it is important to consider it in light of the scale of the Web. Reducing energy consumption by a few percent for billions of users is a significant achievement. Nevertheless, blocking ads and tracking is not a silver bullet. It does not reduce energy consumption on all websites, as ad blockers can introduce bugs or behaviors that lead to higher consumption. Since users visit nearly unique sets of websites [27], ad-blocking software may potentially increase energy consumption for some individuals.

We acknowledge that the top 5,000 websites of the Tranco list may not be representative of the rest of websites in the list nor of all user behaviors nor [27, 61]. Beyond more detailed user interactions on specific websites (e.g., form validation), future work could (a) leverage actual user browsing histories and visit times to obtain more realistic energy consumption profiles, and (b) focus on video-heavy websites (e.g., social media), which offer specific ad formats and capture user attention for significantly longer periods.

Likewise, we observe energy consumption differences between laptop and desktop devices, consistent with prior work reporting variations across operating systems and CPU architectures [31]. Due to POWERAPI limitations, our study focuses on Linux-based devices with integrated-graphics CPUs released between 2017 and 2019. These widely deployed devices remain in use, and their number exceeds that of prior studies [31, 39, 56]. Nevertheless, these consumption variations as well as the rise of the WebGPU API [12] and dedicated GPUs should motivate further research on how hardware architectures and operating systems affect energy use.

Beyond device-level measurements, our study specifically focuses on *client-side* energy consumption, linking privacy to the actual energy cost paid by web users. In line with calls for a greener advertising industry, future work could examine the *server-side* energy consumption of ad delivery infrastructure, particularly for header bidding. While energy consumption is an agnostic metric, its environmental impact varies by region due to differences in energy mixes. Our EU-based crawl could be extended to other jurisdictions to study the carbon emissions associated with ads and tracking.

Finally, at a time when energy savings are critical, it is important to question the place of ads on the Web. Since ads are essential to the Web's economy, they cannot be removed without consequences, but progress should be made to limit their impact on energy consumption. Future work could explore ad schemes designed with energy efficiency in mind. While Google recently backpedaled on the deprecation of third-party cookies [60], the ad industry is currently testing alternatives such as contextual advertising [4], identity graphs on first-party data [5, 6], and interest-based advertising [38, 47, 68]. It remains to be seen whether any of these schemes are less energy intensive.

7 Conclusion

The advertising industry makes up a significant share of Internet traffic due to its prevalence on the Web. In this paper, we analyze the energy consumption of advertisements and associated tracking mechanisms through 724,994 website visits. Overall, we find that blocking ads reduces client-side energy consumption by a median of 9.62%, despite the additional computations performed by browser-based ad blockers. Even without blocking ads, clicking "Accept all" on cookie banners increases median energy consumption by 2.57%. Finally, our analysis of header bidding shows that websites implementing real-time bidding increase client-side energy consumption by a median of 32.88% due to additional CPU operations. Our work provides a large-scale, yet initial, estimate of the actual impact of ads and tracking on web users' energy consumption, highlighting directions for future research.

Ethical Statement

All crawls were conducted with the explicit authorization of the hosting organizations. While the total number of visits may seem large, it amounts to only a few hundred page loads on popular websites over three months; an insignificant volume for high-traffic sites. We accessed only public, unauthenticated pages, and all collected data was encrypted or aggregated.

Acknowledgments

This work has been financially supported by the Agence Nationale de la Recherche through the ANR-21-CE39-0019 FACADES, the ANR-22-PECY-0002 IPoP and the ANR-22-PTCC-0001 SWHSec projects. It was made possible by Software Heritage⁸, the great library of source code, as well as the Laboratoire Haute Sécurité (LHS) of Rennes. This work is supported/is partially funded by Direction Générale de l'Armement through CREACH LABS under the projects PRIVAWE and PRIVABUG.

⁸<https://www.softwareheritage.org/>

References

- [1] 2006. Control Groups – The Linux Kernel Documentation. <https://docs.kernel.org/admin-guide/cgroup-v1/cgroups.html>.
- [2] 2021. Data Centres and Data Transmission Networks. <https://www.iea.org/reports/data-centres-and-data-transmission-networks>.
- [3] 2022. Climate Change Indicators: Greenhouse Gases. <https://www.epa.gov/climate-indicators/greenhouse-gases>.
- [4] 2022. Contextual advertising – Criteo. <https://www.criteo.com/solutions/contextual-advertising/>.
- [5] 2022. Identity Graphs on AWS – AWS. <https://aws.amazon.com/neptune/identity-graphs-on-aws/>.
- [6] 2022. Identity Resolution – LiveRamp. <https://liveramp.com/identity-resolution/>.
- [7] 2022. uBlock Origin. <https://ublockorigin.com/>.
- [8] 2023. CodeCarbon. <https://codecarbon.io/>.
- [9] 2025. Blocklists | AdGuard DNS Knowledge Base. <https://adguard-dns.com/kb/private-dns/setting-up-filtering/blocklists/>.
- [10] 2025. Data Growth Worldwide 2010-2028 - Statista. <https://www.statista.com/statistics/871513/worldwide-data-created/>.
- [11] 2025. Number of internet and social media users worldwide as of February 2025 - Statista. <https://www.statista.com/statistics/617136/digital-population-worldwide/>.
- [12] 2025. WebGPU API - Web APIs | MDN. https://developer.mozilla.org/en-US/docs/Web/API/WebGPU_API.
- [13] Gunes Acar, Christian Eubank, Steven Englehardt, Marc Juarez, Arvind Narayanan, and Claudia Diaz. 2014. The web never forgets: Persistent tracking mechanisms in the wild. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 674–689.
- [14] Syed Suleman Ahmad, Muhammad Daniyal Dar, Muhammad Fareed Zaffar, Narseo Vallina-Rodriguez, and Rishab Nithyanand. 2020. Apophanies or Epiphanies? How Crawlers Impact Our Understanding of the Web. In *Proceedings of The Web Conference 2020*. ACM, Taipei Taiwan, 271–280. doi:10.1145/3366423.3380113
- [15] Nardjes Amieur, Walter Rudametkin, and Oana Goga. 2024. Client-Side and Server-side Tracking on Meta: Effectiveness and Accuracy. *Proceedings on Privacy Enhancing Technologies (PETS) 2024* 1 (2024), 15.
- [16] Abdul Haddi Amjad, Shaoor Munir, Zubair Shafiq, and Muhammad Ali Gulzar. 2024. Blocking Tracking JavaScript at the Function Granularity. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. ACM, Salt Lake City UT USA, 2177–2191. doi:10.1145/3658644.3670329
- [17] Andrea Arcuri and Lionel Briand. 2011. A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In *Proceedings of the 33rd International Conference on Software Engineering*. ACM, Waikiki, Honolulu HI USA, 1–10. doi:10.1145/1985793.1985795
- [18] Muhammad Ahmad Bashir, Sajjad Arshad, Engin Kirda, William Robertson, and Christo Wilson. 2019. A Longitudinal Analysis of the Ads.Txt Standard. In *Proceedings of the Internet Measurement Conference*. ACM, Amsterdam Netherlands, 294–307. doi:10.1145/3355369.3355603
- [19] Muhammad Ahmad Bashir, Sajjad Arshad, William Robertson, and Christo Wilson. 2016. Tracing information flows between ad exchanges using retargeted ads. In *25th USENIX Security Symposium (USENIX Security 16)*. 481–496.
- [20] Paschalis Bekos, Panagiotis Papadopoulos, Evangelos P. Markatos, and Nicolas Kourtellis. 2023. The Hitchhiker's Guide to Facebook Web Tracking with Invisible Pixels and Click IDs. In *Proceedings of the ACM Web Conference 2023 (WWW '23)*. Association for Computing Machinery, New York, NY, USA, 2132–2143. doi:10.1145/3543507.3583311
- [21] Rob Bonta. 2022. California consumer privacy act (CCPA). Retrieved from State of California Department of Justice: <https://oag.ca.gov/privacy/ccpa> (2022).
- [22] Ahmed Bouhoula, Karel Kubicek, Amit Zac, Carlos Cotrini, and David Basin. 2024. Automated {Large-Scale} Analysis of Cookie Notice Compliance. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1723–1739.
- [23] Emile Cadorel and Dimitris Saingre. 2024. A Protocol to Assess the Accuracy of Process-Level Power Models. In *IEEE International Conference on Cluster Computing, CLUSTER 2024, Kobe, Japan, September 24-27, 2024*. IEEE, 74–84. doi:10.1109/CLUSTER59578.2024.00014
- [24] Xiaomeng Chen, Abhilash Jindal, and Y. Charlie Hu. 2013. How Much Energy Can We Save from Prefetching Ads?: Energy Drain Analysis of Top 100 Apps. In *Proceedings of the Workshop on Power-Aware Computing and Systems*. ACM, Farmington Pennsylvania, 1–5. doi:10.1145/2525526.2525848
- [25] Salim Chouaki, Oana Goga, Hamed Haddadi, and Peter Snyder. 2023. Understanding the Privacy Risks of Popular Search Engine Advertising Systems. In *Proceedings of the 2023 ACM on Internet Measurement Conference (Montreal QC, Canada) (IMC '23)*. Association for Computing Machinery, New York, NY, USA, 370–382. doi:10.1145/3618257.3624823
- [26] John Cook, Rishab Nithyanand, and Zubair Shafiq. 2020. Inferring Tracker-Advertiser Relationships in the Online Advertising Ecosystem using Header Bidding. *Proceedings on Privacy Enhancing Technologies* 1 (2020), 65–82.
- [27] Kyle Crichton, Nicolas Christin, and Lorrie Faith Cranor. 2022. How Do Home Computer Users Browse the Web? *ACM Transactions on the Web* 16, 1 (Feb. 2022), 1–27. doi:10.1145/3473343
- [28] Salvatore D'Ambrosio, Salvatore De Pasquale, Gerardo Iannone, Delfina Malandrino, Alberto Negro, Giovanni Patimo, Vittorio Scarano, and Raffaele Spinelli. 2016. Energy Consumption and Privacy in Mobile Web Browsing: Individual Issues and Connected Solutions. *Sustainable Computing: Informatics and Systems* 11 (Sept. 2016), 63–79. doi:10.1016/j.suscom.2016.02.003
- [29] Guillaume Fieni, Daniel Romero Acero, Pierre Rust, and Romain Rouvoy. 2024. PowerAPI: A Python Framework for Building Software-Defined Power Meters. *Journal of Open Source Software* 9, 98 (2024), 6670.
- [30] Guillaume Fieni, Romain Rouvoy, and Lionel Seinturier. 2020. SmartWatts: Self-calibrating Software-Defined Power Meter for Containers. In *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. IEEE, 479–488.
- [31] José González-Cabañas, Patricia Callejo, Rubén Cuevas, Steffen Svartberg, Tommy Torjesen, Ángel Cuevas, Antonio Pastor, and Mikko Kotila. 2023. CarbonTag: A Browser-Based Method for Approximating Energy Consumption of Online Ads. *IEEE Transactions on Sustainable Computing* (2023).
- [32] Colin M. Gray, Cristiana Santos, Natalia Bielova, Michael Toth, and Damian Clifford. 2021. Dark Patterns and the Legal Requirements of Consent Banners: An Interaction Criticism Perspective. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, Yokohama Japan, 1–18. doi:10.1145/3411764.3445779
- [33] IAB. [n. d.]. OpenRTB (Real-Time Bidding).
- [34] IAB. 2024. Internet Advertising Revenue Report: Full Year 2024. <https://www.iab.com/insights/internet-advertising-revenue-report-full-year-2024/>.
- [35] Intel Corporation. 2025. Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4. Intel Corporation. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html> Updated October 29, 2025.
- [36] Mathilde Jay, Vladimir Ostapenko, Laurent Lefèvre, Denis Trystram, Anne-Cécile Orgerie, and Benjamin Fichel. 2023. An Experimental Comparison of Software-Based Power Meters: Focus on CPU and GPU. In *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 106–118.
- [37] Nikhil Jha, Martino Trevisan, Luca Vassio, and Marco Mellia. 2022. The Internet with privacy policies: Measuring the Web upon consent. *ACM Transactions on the Web (TWEB)* 16, 3 (2022), 1–24.
- [38] John Wilander. 2021. Introducing Private Click Measurement, PCM – WebKit Blog. <https://webkit.org/blog/11529/introducing-private-click-measurement-pcm/>.
- [39] Khan Awais Khan, Mohammad Tariq Iqbal, and Mohsin Jamil. 2024. Impact of Ad Blockers on Computer Power Consumption While Web Browsing: A Comparative Analysis. *European Journal of Electrical Engineering and Computer Science* 8, 5 (2024), 18–24.
- [40] Atefeh Khosravi, Lachlan LH Andrew, and Rajkumar Buyya. 2017. Dynamic VM placement method for minimizing energy and carbon cost in geographically distributed cloud data centers. *IEEE Transactions on Sustainable Computing* 2, 2 (2017), 183–196.
- [41] Atefeh Khosravi and Rajkumar Buyya. 2018. Energy and carbon footprint-aware management of geo-distributed cloud data centers: A taxonomy, state of the art, and future directions. *Sustainable Development: Concepts, Methodologies, Tools, and Applications* (2018), 1456–1475.
- [42] Atefeh Khosravi, Saurabh Kumar Garg, and Rajkumar Buyya. 2013. Energy and carbon-efficient placement of virtual machines in distributed cloud data centers. In *European Conference on Parallel Processing*. Springer, 317–328.
- [43] Tomer Laor, Naif Mehanna, Antonin Durey, Vitaly Dyadyuk, Pierre Laperdrix, Clémentine Maurice, Yossi Oren, Romain Rouvoy, Walter Rudametkin, and Yuval Yarom. 2022. DRAWNAPART: A Device Identification Technique based on Remote GPU Fingerprinting. *arXiv preprint arXiv:2201.09956* (2022).
- [44] Pierre Laperdrix, Natalia Bielova, Benoit Baudry, and Gildas Avoine. 2020. Browser fingerprinting: A survey. *ACM Transactions on the Web (TWEB)* 14, 2 (2020), 1–33.
- [45] Zengrui Liu, Umar Iqbal, and Nitesh Saxena. 2024. Opted Out, Yet Tracked: Are Regulations Enough to Protect Your Privacy? *Proceedings on Privacy Enhancing Technologies* 2024, 1 (Jan. 2024), 280–299. doi:10.56553/popets-2024-0016
- [46] Daniela Lopes, Daniel Castro, Diogo Barradas, and Nuno Santos. 2023. TIGER: Tor Traffic Generator for Realistic Experiments. In *Proceedings of the 22nd Workshop on Privacy in the Electronic Society*. ACM, Copenhagen Denmark, 147–152. doi:10.1145/3603216.3624960
- [47] Martin Thomson. 2022. Privacy Preserving Attribution for Advertising – Mozilla Blog. <https://blog.mozilla.org/en/mozilla/privacy-preserving-attribution-for-advertising/>.
- [48] Asya Mitseva and Andriy Panchenko. 2024. Stop, Don't Click Here Anymore: Boosting Website Fingerprinting by Considering Sets of Subpages. In *33rd USENIX Security Symposium (USENIX Security 24)*. 4139–4156.
- [49] Keaton Mowery and Hovav Shacham. 2012. Pixel perfect: Fingerprinting canvas in HTML5. *Proceedings of W2SP 2012* (2012).
- [50] Maaz Bin Musa and Rishab Nithyanand. 2022. ATOM: Ad-network Tomography. *Proceedings on Privacy Enhancing Technologies* 4 (2022), 295–313.

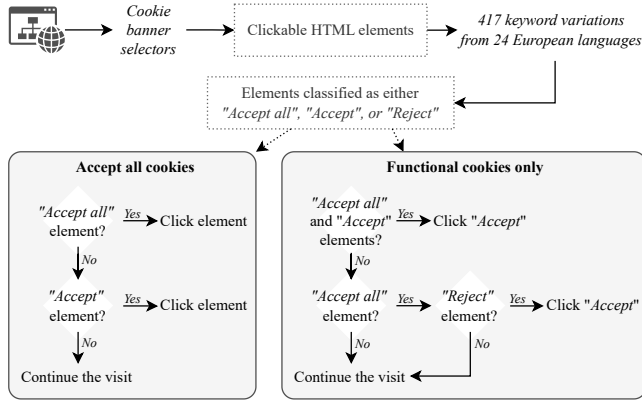


Figure 9: Cookie handling process.

- [51] Lukasz Olejnik, Tran Minh-Dung, and Claude Castelluccia. 2013. Selling off privacy at auction. (2013).
- [52] Michalis Pachilakis, Panagiotis Papadopoulos, Evangelos P Markatos, and Nicolas Kourtellis. 2019. No more chasing waterfalls: a measurement study of the header bidding ad-ecosystem. In *Proceedings of the Internet Measurement Conference*. 280–293.
- [53] Andriy Panchenko, Fabian Lanze, Jan Pennekamp, Thomas Engel, Andreas Zinnen, Martin Henze, and Klaus Wehrle. 2016. Website Fingerprinting at Internet Scale.. In *NDSS*.
- [54] Nikolaos Pantelaio and Alexandros Kapravelos. 2024. Manifest V3 Unveiled: Navigating the New Era of Browser Extensions. doi:10.14722/madweb.2024.23080 arXiv:2404.08310 [cs]
- [55] Panagiotis Papadopoulos, Nicolas Kourtellis, and Evangelos Markatos. 2019. Cookie synchronization: Everything you always wanted to know but were afraid to ask. In *The World Wide Web Conference*. 1432–1442.
- [56] Joshua M Pearce. 2020. Energy conservation with open source ad blockers. *Technologies* 8, 2 (2020), 18.
- [57] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. 2018. Tranco: A research-oriented top sites ranking hardened against manipulation. *arXiv preprint arXiv:1806.01156* (2018).
- [58] Zahra Pooranian, Mauro Conti, Hamed Haddadi, and Rahim Tafazolli. 2021. Online Advertising Security: Issues, Taxonomy, and Future Directions. *IEEE Communications Surveys & Tutorials* 23, 4 (2021), 2494–2524.
- [59] Enric Pujol, Oliver Hohlfeld, and Anja Feldmann. 2015. Annoyed users: Ads and ad-block usage in the wild. In *Proceedings of the 2015 Internet Measurement Conference*. 93–106.
- [60] Emma Roth. 2025. Google Is Scrapping Its Planned Changes for Third-Party Cookies in Chrome. <https://www.theverge.com/news/653964/google-privacy-sandbox-plans-scrapped-third-party-cookies>.
- [61] Kimberly Ruth, Aurore Fass, Jonathan Azose, Mark Pearson, Emma Thomas, Caitlin Sadowski, and Zakir Durumeric. 2022. A World Wide View of Browsing the World Wide Web. In *Proceedings of the 22nd ACM Internet Measurement Conference*. ACM, Nice France, 317–336. doi:10.1145/3517745.3561418
- [62] Kimberly Ruth, Deepak Kumar, Brandon Wang, Luke Valenta, and Zakir Durumeric. 2022. Toppling Top Lists: Evaluating the Accuracy of Popular Website Lists. In *Proceedings of the 22nd ACM Internet Measurement Conference*. ACM, Nice France, 374–387. doi:10.1145/3517745.3561444
- [63] Nayanamana Samarasinghe and Mohammad Mannan. 2019. Towards a global perspective on web tracking. *Computers & Security* 87 (2019), 101569.
- [64] Iskander Sanchez-Rola, Xabier Ugarte-Pedrero, Igor Santos, and Pablo G Bringas. 2017. The web is watching you: A comprehensive review of web-tracking techniques and countermeasures. *Logic Journal of the IGPL* 25, 1 (2017), 18–29.
- [65] Danielle Smith. 2017. What Is Header Bidding? | Header Bidding Definition | Lotame.
- [66] Statista. 2021. Adblocking penetration rate in 2021. <https://www.statista.com/statistics/351862/adblocking-usage/>.
- [67] Publications Office of the European Union. 2016. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data, and Repealing Directive 95/46/EC (General Data Protection Regulation) (Text with EEA Relevance). <https://op.europa.eu/en/publication-detail/-/publication/3e485e15-11bd-11e6-ba9a-01aa75ed71a1/language-en>.

- [68] Vinay Goel. 2022. Get to know the new Topics API for Privacy Sandbox – Google Blog. <https://blog.google/products/chrome/get-know-new-topics-api-privacy-sandbox/>.
- [69] Brian J Watson, Amip J Shah, Manish Marwah, Cullen E Bash, Ratnesh K Sharma, Christopher E Hoover, Tom W Christian, and Chandrakant D Patel. 2009. Integrated design and management of a sustainable data center. In *International Electronic Packaging Technical Conference and Exhibition*, Vol. 43604. 635–644.

A Cookie handling process

Figure 9 details how our automation process handles cookies on each visited website. To interact with banners, we first extract the cookie banner from the DOM using the *EasyList Cookie List*, a well-known filter list containing selectors for numerous cookie banners. Second, we analyze the banner’s content to detect interactive elements (e.g., buttons, links). These elements are classified based on 417 keyword variations of consent-related phrases across all 24 official languages of the European Union. We focus on EU languages because websites targeting European users are more likely to implement cookie banners due to the GDPR, which requires consent to target and track users [67]. Third, improving upon the work of Jha *et al.* [37], we distinguish between *accept*, *accept all*, and *reject* actions. We select the appropriate action according to the cookie setting used for the current crawl and simulate a click on the corresponding HTML element. Although the *EasyList Cookie List* is well-maintained, it may miss certain websites, and others may not display banners at all. Ultimately, we successfully interact with the cookie banner (i.e., click an element) on approximately 70% of the crawled websites.

In any case, since websites *should* [45] refrain from tracking users before receiving consent, minimizing cookie acceptance provides a conservative lower-bound estimate of the impact of refusing consent on energy consumption.

B Client devices

Table 2 details the client devices on which we measured energy consumption. Only the *Intel NUC 9i7QNB* is a desktop device. Due to the requirements of POWERAPI [29] and limited support, more recent devices could not be enrolled in the experiments.

Table 2: Client devices used for measurements.

Device name	CPU	RAM (Go)	TDP (W)	Launch year
Intel NUC 9i7QNB	i7-9570H	32	45	2019
Dell Latitude 7490	i7-8650U	32	15	2017
Dell XPS 13 7390	i7-10510U	16	15	2019
Lenovo Thinkpad T480	i7-8550U	16	15	2017

C Statistical tests

Following best practices, we compare the paired value distributions obtained through our experiments via a one-sample Wilcoxon signed-rank test, reporting the associated p-value, and Cohen’s d [17] (see Tables 3, 4, 5, 6, and 7). We observe a small to medium (yet statistically significant) shift between targeted distributions compared to the baseline. Finally, we observe an extremely weak correlation between energy consumption variations and the rank of websites in the Tranco list (Spearman 0.017, p-value < 0.001).

Table 7: Statistical tests for Figure 8.

Device name	Cohen's d	p-value
Intel NUC 9i7QNB	0.074	0.00
Dell Latitude 7490	-0.148	0.00
Lenovo Thinkpad T480	-0.075	0.00
Dell XPS 13 7390	-0.580	0.00

Table 3: Statistical tests for Figure 4.

Crawl variant	Cohen's d	p-value
Browser-based ad blocker with <i>Accept all cookies</i>	-0.204	0.00
Browser-based ad blocker with <i>Functional cookies only</i>	-0.115	0.00

Table 4: Statistical tests for Figure 5.

Category	Cohen's d	p-value
News	-0.581	0.00
Shopping	-0.401	0.00
Pornography	-0.319	0.00
Business/Economy	-0.248	0.00
Education	-0.175	0.00
Entertainment	-0.229	0.00
Others	-0.203	0.00
Technology/Internet	-0.142	0.00
Government/Legal	-0.077	0.02
Search Engines/Portals	0.149	0.00

Table 5: Statistical tests for Figure 6.

Bidding setup	Cohen's d	p-value
No bidding detected	-0.130	0.00
ads.txt only	-0.395	0.00
prebid.js only	-0.487	0.05
Both	-1.258	0.00

Table 6: Statistical tests for Figure 7.

Crawl variant	Cohen's d	p-value
uBlock Origin with <i>Accept all cookies</i>	-0.270	0.00
uBlock Origin Lite with <i>Accept all cookies</i>	-0.112	0.00
Network-based ad blocker with <i>Accept all cookies</i>	-0.176	0.00